

Phase 3: AVL C Implementation and Large-Tree Rotation Formulas

1. Where We Are Now

Before Phase 3, you already learned two big things.

From Phase 1

You learned how to **detect imbalance** in an AVL tree.

An AVL tree is a Binary Search Tree, or BST, with one extra rule:

Every node must stay balanced.

A BST follows this rule:

left subtree values < node value < right subtree values

An AVL tree also checks the **balance factor** of every node:

Balance Factor = height(left subtree) - height(right subtree)

The valid balance factors are:

-1, 0, +1

The invalid balance factors are:

+2, -2, +3, -3, ...

So Phase 1 taught you this question:

Is the tree balanced or unbalanced?

From Phase 2

You learned how to **fix imbalance** using rotations.

There are four insertion imbalance cases:

Case	Shape	Rotation Type
LL	Left-left straight line	Single rotation
RR	Right-right straight line	Single rotation
LR	Left-right zig-zag	Double rotation
RL	Right-left zig-zag	Double rotation

Phase 2 taught you this question:

If the tree is unbalanced, which rotation fixes it?

What Phase 3 Adds

Phase 3 connects the theory to real C code.

In Phase 2, rotations looked like drawings.

In Phase 3, rotations become **pointer operations**.

The main idea is:

Phase 3 teaches how to implement AVL insertion safely in C, especially when rotated nodes already have subtrees attached.

2. Main Goal of Phase 3

The main goal of Phase 3 is to answer this question:

How do we turn AVL rotation theory into correct C code?

By the end of this phase, you should understand:

- How an AVL node is stored in C.
- Why each node stores height.
- How to calculate node height.
- How to calculate balance factor.
- How recursive AVL insertion works.
- Why height updates happen while recursion returns.
- How the program chooses LL, RR, LR, or RL.
- How rotations preserve attached subtrees.
- What orphan subtrees are.

- Why rotations must return the new subtree root.
- Why rotations take constant time.

3. The AVL Node in C

A normal BST node usually stores:

- data
- left child pointer
- right child pointer

An AVL node stores one extra thing:

- height

In Phase 3, the C node is written like this:

```
struct Node {
    struct Node *lchild;
    int data;
    int height;
    struct Node *rchild;
};
```

Meaning of Each Field

Field	Meaning
<code>lchild</code>	Pointer to the left child
<code>data</code>	The value stored in the node
<code>height</code>	Height of the subtree rooted at this node
<code>rchild</code>	Pointer to the right child

You can imagine one node like this:

```

      data
     /
lchild   rchild
```

The `height` field is very important because AVL trees need height to calculate balance factor.

4. Why AVL Nodes Store Height

Balance factor needs height:

```
BF = height(left subtree) - height(right subtree)
```

If height is stored inside each node, the program can read the height quickly.

Reading stored height takes:

```
O(1)
```

That means constant time.

If the height was not stored, the program might need to scan down the subtree again and again to calculate height. That would make AVL insertion slower.

So AVL nodes store height to make balance checking fast.

Memory line:

AVL nodes store height so balance factor can be calculated quickly.

5. Height Rules Used in This Curriculum

This curriculum uses these height rules:

```
NULL child height = 0  
leaf node height = 1
```

A leaf node is a node with no children.

Example:

```
10
```

Node **10** has no left child and no right child.

So:

```
left height = 0
right height = 0
height(10) = 1
```

The height formula is:

```
height(node) = 1 + max(height(left child), height(right child))
```

6. Helper Function: NodeHeight

The `NodeHeight` function calculates the height of a node using the heights of its children.

```
int NodeHeight(struct Node *p) {
    int hl = 0;
    int hr = 0;

    if (p != NULL && p->lchild != NULL)
        hl = p->lchild->height;

    if (p != NULL && p->rchild != NULL)
        hr = p->rchild->height;

    return 1 + (hl > hr ? hl : hr);
}
```

What the Variables Mean

Variable	Meaning
<code>p</code>	The current node
<code>hl</code>	Height of the left child
<code>hr</code>	Height of the right child

Why `hl` and `hr` Start as 0

They start as `0` because a missing child has height `0`.

```
int hl = 0;  
int hr = 0;
```

This means:

Assume both child heights are 0 unless real children exist.

Why We Check for `NULL`

This line checks whether the left child exists:

```
if (p != NULL && p->lchild != NULL)
```

Then the program safely reads:

```
hl = p->lchild->height;
```

This is important because you must not read height from a missing child.

Wrong idea:

```
hl = p->lchild->height;
```

This is dangerous if `p->lchild` is `NULL`.

Correct idea:

```
if (p != NULL && p->lchild != NULL)  
    hl = p->lchild->height;
```

Memory line:

Always check that a pointer is not `NULL` before using it.

7. Helper Function: `BalanceFactor`

The balance factor tells whether a node is balanced.

```

int BalanceFactor(struct Node *p) {
    int hl = 0;
    int hr = 0;

    if (p != NULL && p->lchild != NULL)
        hl = p->lchild->height;

    if (p != NULL && p->rchild != NULL)
        hr = p->rchild->height;

    return hl - hr;
}

```

The formula is:

Balance Factor = left height - right height

Short form:

BF = hl - hr

Meaning of Balance Factor Values

Balance Factor	Meaning	AVL Status
+1	Left side is taller by 1	Valid
0	Both sides have same height	Valid
-1	Right side is taller by 1	Valid
+2	Left side is too tall	Invalid
-2	Right side is too tall	Invalid

So:

BF = +2 means left-heavy problem.
 BF = -2 means right-heavy problem.

8. Creating a New Node

When inserting into an AVL tree, sometimes we reach a `NULL` position. That means the new value should be inserted there.

A new node is created like this:

```
struct Node *CreateNode(int key) {  
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));  
  
    t->data = key;  
    t->height = 1;  
    t->lchild = NULL;  
    t->rchild = NULL;  
  
    return t;  
}
```

Why Height Starts at 1

A new node has no children, so it is a leaf.

In this curriculum:

```
leaf height = 1
```

So:

```
t->height = 1;
```

Why Children Start as `NULL`

A new node has no children yet.

So:

```
t->lchild = NULL;  
t->rchild = NULL;
```

9. AVL Insertion Begins Like BST Insertion

AVL insertion starts the same way as normal BST insertion.

If the new key is smaller than the current node, go left.

If the new key is greater than the current node, go right.

If the key is equal, this beginner version ignores duplicates.

Basic insertion logic:

```
if (key < p->data) {
    p->lchild = Insert(p->lchild, key);
} else if (key > p->data) {
    p->rchild = Insert(p->rchild, key);
} else {
    return p;
}
```

Why the Recursive Result Is Assigned Back

Look at this line:

```
p->lchild = Insert(p->lchild, key);
```

This is important.

The recursive call may return:

- the same child root, or
- a new child root after rotation

So we must assign the result back to `p->lchild`.

Same idea for the right child:

```
p->rchild = Insert(p->rchild, key);
```

Memory line:

Always store the returned subtree root after recursive insertion.

10. Why AVL Insertion Is Recursive

AVL insertion uses recursion because we need to go down to the correct position first, then fix the tree while coming back up.

The process is:

1. Go down like normal BST insertion.
2. Insert the new node.
3. Return back upward.
4. Update height at each ancestor.
5. Check balance factor at each ancestor.
6. Rotate if needed.

The important part happens during the return phase.

Memory line:

AVL insertion fixes the tree while recursion returns upward.

11. Why Height Updates Happen After Recursive Insertion

After inserting a new node, the height of some ancestors may change.

Example:

Before inserting 10:

20

After inserting 10:

20
/
10

Node 20 now has a left child, so its height may change.

That is why insertion does this after the recursive call:

```
p->height = NodeHeight(p);
```

or in some versions:

```
UpdateHeight(p);
```

You must update height before checking balance factor.

Wrong order:

```
Check balance first, then update height.
```

Correct order:

```
Update height first, then check balance.
```

Why?

Because balance factor uses height.

If the height is old, the balance factor may be wrong.

Memory line:

Update height before checking balance factor.

12. Rotation Selection After Insertion

After updating height, the program checks if the current node is imbalanced.

There are two main imbalance signs:

```
BF = +2 means the left side is too tall.  
BF = -2 means the right side is too tall.
```

But that is not enough to choose the rotation.

We also need to inspect the child balance factor.

13. Rotation Selection Table

Parent Balance Factor	Child Balance Factor	Case	Rotation
+2	≥ 0 on left child	LL	LLRotation
+2	< 0 on left child	LR	LRRotation
-2	≤ 0 on right child	RR	RRRotation
-2	> 0 on right child	RL	RLRotation

Code Version

```
if (BalanceFactor(p) == 2 && BalanceFactor(p->lchild) >= 0)
    return LLRotation(p);

if (BalanceFactor(p) == 2 && BalanceFactor(p->lchild) < 0)
    return LRRotation(p);

if (BalanceFactor(p) == -2 && BalanceFactor(p->rchild) <= 0)
    return RRRotation(p);

if (BalanceFactor(p) == -2 && BalanceFactor(p->rchild) > 0)
    return RLRotation(p);
```

Beginner Meaning

If the current node is too heavy on the left, check the left child.

Parent BF = +2

Then:

left child BF ≥ 0 means LL
left child BF < 0 means LR

If the current node is too heavy on the right, check the right child.

Parent BF = -2

Then:

```
right child BF <= 0 means RR
right child BF > 0 means RL
```

14. Why Child Balance Factor Helps

Imagine the parent is left-heavy:

```
BF(parent) = +2
```

This only tells us:

```
The problem is somewhere on the left side.
```

But there are two left-side cases:

```
LL: left then left
LR: left then right
```

So we inspect the left child.

If the left child is also left-heavy or balanced, it is LL.

If the left child is right-heavy, it is LR.

Same idea for right-heavy cases.

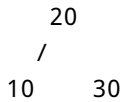
15. Large-Tree Rotations

In Phase 2, rotations were shown using only three nodes.

Example LL:

```
    30
   /
  20
 /
10
```

After rotation:



This is simple because there are no extra subtrees attached.

But real AVL trees can be larger.

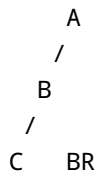
A node involved in rotation may already have a subtree attached.

That attached subtree must not be lost.

16. What Is an Orphan Subtree?

An orphan subtree is a subtree that could get disconnected during pointer changes if we do not save it.

Example:



Here:

- A is the imbalanced node.
- B is the left child of A.
- C is the left child of B.
- BR is the right subtree of B.

When we do LL rotation, B moves up and A moves down.

But BR is in the way.

If we are not careful, BR can be lost.

So we save and reattach it.

Memory line:

Orphan subtrees are subtrees that must be saved before pointer changes overwrite them.

17. Large-Tree LL Rotation Formula

Before LL Rotation

```
  A
 /
B
 /
C  BR
```

After LL Rotation

```
  B
 /
C  A
   /
  BR
```

Why Does **BR** Become the Left Child of **A**?

Because of BST ordering.

Before rotation:

```
B < BR < A
```

That means all values in **BR** are:

- greater than **B**
- smaller than **A**

After **A** moves down to the right of **B**, **BR** belongs on the left side of **A**.

So the correct reattachment is:

```
A->left = BR
```

18. LL Rotation in C

```
struct Node *LLRotation(struct Node *p) {  
    struct Node *pl = p->lchild;  
    struct Node *plr = pl->rchild;  
  
    pl->rchild = p;  
    p->lchild = plr;  
  
    p->height = NodeHeight(p);  
    pl->height = NodeHeight(pl);  
  
    return pl;  
}
```

Variable Meanings

Variable	Meaning
p	Old root of the imbalanced subtree
pl	Left child of p
plr	Right subtree of pl

Before

```
    p  
   /\br/>  pl  
   /\br/>  plr
```

Pointer Change 1

```
pl->rchild = p;
```

This makes p move down to the right of pl.

Pointer Change 2

```
p->lchild = plr;
```


This reattaches the orphan subtree `plr` as the left child of `p`.

After

```
pl
  p
  /
plr
```

Height Updates

```
p->height = NodeHeight(p);
pl->height = NodeHeight(pl);
```

We update `p` first because `p` is now lower.

Then we update `pl` because `pl` is now the new root.

Memory line:

After rotation, update the lower node first, then update the new root.

Return Value

```
return pl;
```

We return `pl` because `pl` is now the new root of this local subtree.

19. Large-Tree RR Rotation Formula

RR is the mirror version of LL.

Before RR Rotation

```
A
  B
  /
BL C
```

After RR Rotation

```
    B
   /
  A  C
   \
    BL
```

Why Does BL Become the Right Child of A ?

Because of BST ordering.

Before rotation:

```
A < BL < B
```

So after A moves down, BL belongs as the right child of A.

20. RR Rotation in C

```
struct Node *RRRotation(struct Node *p) {
    struct Node *pr = p->rchild;
    struct Node *prl = pr->lchild;

    pr->lchild = p;
    p->rchild = prl;

    p->height = NodeHeight(p);
    pr->height = NodeHeight(pr);

    return pr;
}
```

Variable Meanings

Variable	Meaning
p	Old root of the imbalanced subtree
pr	Right child of p
prl	Left subtree of pr

Before

```
p
  pr
 /
prl
```

Pointer Change 1

```
pr->lchild = p;
```

This makes `p` move down to the left of `pr`.

Pointer Change 2

```
p->rchild = prl;
```

This reattaches the orphan subtree `prl` as the right child of `p`.

Return Value

```
return pr;
```

We return `pr` because `pr` is now the new local root.

21. Large-Tree LR Rotation Formula

LR is a zig-zag case.

In LR, the middle node becomes the new root.

Before LR Rotation

```
  A
 /
B
  C
```

```

      /
CL   CR

```

After LR Rotation

```

      C
      /
     B  A
    \  /
   CL CR

```

A clearer expanded version:

```

      C
      /
     B  A
    \  /
   CL CR

```

Meaning:

```

B->right = CL
A->left  = CR
C->left  = B
C->right = A

```

Why Does CL Become B->right ?

Because values in CL are:

```

B < CL < C

```

So CL belongs on the right side of B.

Why Does CR Become A->left ?

Because values in CR are:

```

C < CR < A

```

So CR belongs on the left side of A.

Memory line:

In LR, save C's left and right subtrees before moving C to the top.

22. LR Rotation in C

```
struct Node *LRRotation(struct Node *p) {
    struct Node *pl = p->lchild;
    struct Node *plr = pl->rchild;

    pl->rchild = plr->lchild;
    p->lchild = plr->rchild;

    plr->lchild = pl;
    plr->rchild = p;

    pl->height = NodeHeight(pl);
    p->height = NodeHeight(p);
    plr->height = NodeHeight(plr);

    return plr;
}
```

Variable Meanings

Variable	Meaning
<code>p</code>	Old root of the imbalanced subtree
<code>pl</code>	Left child of <code>p</code>
<code>plr</code>	Right child of <code>pl</code> , the middle node

Before

```
    p
   /
  pl

    plr
   /
left right
```

Pointer Change 1

```
pl->rchild = plr->lchild;
```

This attaches `plr`'s left subtree to the right side of `pl`.

Pointer Change 2

```
p->lchild = plr->rchild;
```

This attaches `plr`'s right subtree to the left side of `p`.

Pointer Change 3

```
plr->lchild = pl;
```

This makes `pl` the left child of `plr`.

Pointer Change 4

```
plr->rchild = p;
```

This makes `p` the right child of `plr`.

After

```
    plr
   /
  pl  p
 \  /
left right
```

Height Updates

```
pl->height = NodeHeight(pl);
p->height = NodeHeight(p);
plr->height = NodeHeight(plr);
```

We update the lower nodes first:

```
pl first
p second
plr last
```

Why?

Because `plr` is the new root, so its height depends on the updated heights of `pl` and `p`.

Return Value

```
return plr;
```

We return `plr` because `plr` is now the new local root.

Memory line:

In LR, `plr` becomes the new root.

23. Large-Tree RL Rotation Formula

RL is the mirror version of LR.

In RL, the middle node also becomes the new root.

Before RL Rotation

```
  A
   /
  B
 /
C
/
CL CR
```

After RL Rotation

```
  C
 /
A  B
```

\ /
CL CR

Meaning:

```
A->right = CL  
B->left  = CR  
C->left  = A  
C->right = B
```

Why Does CL Become A->right ?

Because values in CL are:

A < CL < C

So CL belongs on the right side of A.

Why Does CR Become B->left ?

Because values in CR are:

C < CR < B

So CR belongs on the left side of B.

24. RL Rotation in C

```
struct Node *RLRotation(struct Node *p) {  
    struct Node *pr = p->rchild;  
    struct Node *prl = pr->lchild;  
  
    pr->lchild = prl->rchild;  
    p->rchild = prl->lchild;  
  
    prl->rchild = pr;  
    prl->lchild = p;  
  
    pr->height = NodeHeight(pr);  
    p->height = NodeHeight(p);  
    prl->height = NodeHeight(prl);  
}
```



```
    return prl;  
}
```

Variable Meanings

Variable	Meaning
<code>p</code>	Old root of the imbalanced subtree
<code>pr</code>	Right child of <code>p</code>
<code>prl</code>	Left child of <code>pr</code> , the middle node

Before

```
p  
  pr  
  /  
 prl  
  /  
left right
```

Pointer Change 1

```
pr->lchild = prl->rchild;
```

This attaches `prl`'s right subtree to the left side of `pr`.

Pointer Change 2

```
p->rchild = prl->lchild;
```

This attaches `prl`'s left subtree to the right side of `p`.

Pointer Change 3

```
prl->rchild = pr;
```

This makes `pr` the right child of `prl`.

Pointer Change 4

```
prl->lchild = p;
```

This makes `p` the left child of `prl`.

After

```
    prl
   /
  p   pr
 \   /
left right
```

Return Value

```
return prl;
```

We return `prl` because `prl` is now the new local root.

Memory line:

In RL, `prl` becomes the new root.

25. Summary of the Four Rotations in C

Case	Main Nodes	New Root	What Happens
LL	<code>p</code> , <code>pl</code> , <code>plr</code>	<code>pl</code>	Left child moves up
RR	<code>p</code> , <code>pr</code> , <code>prl</code>	<code>pr</code>	Right child moves up
LR	<code>p</code> , <code>pl</code> , <code>plr</code>	<code>plr</code>	Middle node moves up
RL	<code>p</code> , <code>pr</code> , <code>prl</code>	<code>prl</code>	Middle node moves up

Memory lines:

```
LL: left child moves up.
RR: right child moves up.
```

LR: middle node moves up.
RL: middle node moves up.

26. Why Rotations Preserve BST Order

A rotation changes the shape of the tree, but it does not change the sorted order of the values.

Example:

Before LL rotation:

```
    30
   /
  20
 /
10
```

After LL rotation:

```
    20
   /
  10  30
```

The inorder traversal before rotation is:

10, 20, 30

The inorder traversal after rotation is still:

10, 20, 30

So the BST rule is preserved.

In large-tree rotations, orphan subtrees are reattached based on value ranges.

Example LL:

$B < BR < A$

So **BR** becomes the left child of **A** after rotation.

Memory line:

Rotations change shape, not sorted order.

27. Why Every Rotation Returns a Pointer

Every rotation function returns the new root of the local subtree.

Example LL:

Before:

```
30 is the local root.
```

After:

```
20 is the local root.
```

So the function must return the new root.

Example:

```
return pl;
```

If we return the old root by mistake, the parent node will still point to the wrong place.

Memory line:

After rotation, return the new local root.

28. Complete AVL Insertion Code

This is a complete beginner-friendly AVL insertion program in C.

```
#include <stdio.h>
#include <stdlib.h>
```

```

typedef struct Node {
    struct Node *lchild;
    int data;
    int height;
    struct Node *rchild;
} Node;

int max(int a, int b) {
    return (a > b) ? a : b;
}

int NodeHeight(Node *p) {
    if (p == NULL)
        return 0;

    return p->height;
}

int BalanceFactor(Node *p) {
    if (p == NULL)
        return 0;

    return NodeHeight(p->lchild) - NodeHeight(p->rchild);
}

void UpdateHeight(Node *p) {
    if (p != NULL) {
        p->height = 1 + max(NodeHeight(p->lchild), NodeHeight(p->rchild));
    }
}

Node *CreateNode(int key) {
    Node *t = (Node *)malloc(sizeof(Node));

    t->data = key;
    t->height = 1;
    t->lchild = NULL;
    t->rchild = NULL;

    return t;
}

Node *LLRotation(Node *p) {
    Node *pl = p->lchild;
    Node *plr = pl->rchild;

    pl->rchild = p;
    p->lchild = plr;
}

```

```

        UpdateHeight(p);
        UpdateHeight(pl);

        return pl;
    }

Node *RRRotation(Node *p) {
    Node *pr = p->rchild;
    Node *prl = pr->lchild;

    pr->lchild = p;
    p->rchild = prl;

    UpdateHeight(p);
    UpdateHeight(pr);

    return pr;
}

Node *LRRotation(Node *p) {
    Node *pl = p->lchild;
    Node *plr = pl->rchild;

    pl->rchild = plr->lchild;
    p->lchild = plr->rchild;

    plr->lchild = pl;
    plr->rchild = p;

    UpdateHeight(pl);
    UpdateHeight(p);
    UpdateHeight(plr);

    return plr;
}

Node *RLRotation(Node *p) {
    Node *pr = p->rchild;
    Node *prl = pr->lchild;

    pr->lchild = prl->rchild;
    p->rchild = prl->lchild;

    prl->rchild = pr;
    prl->lchild = p;

    UpdateHeight(pr);

```

```

    UpdateHeight(p);
    UpdateHeight(prl);

    return prl;
}

Node *Insert(Node *p, int key) {
    if (p == NULL) {
        return CreateNode(key);
    }

    if (key < p->data) {
        p->lchild = Insert(p->lchild, key);
    } else if (key > p->data) {
        p->rchild = Insert(p->rchild, key);
    } else {
        return p; // duplicate keys are ignored
    }

    UpdateHeight(p);

    if (BalanceFactor(p) == 2 && BalanceFactor(p->lchild) >= 0) {
        return LLRotation(p);
    }

    if (BalanceFactor(p) == 2 && BalanceFactor(p->lchild) < 0) {
        return LRRotation(p);
    }

    if (BalanceFactor(p) == -2 && BalanceFactor(p->rchild) <= 0) {
        return RRRotation(p);
    }

    if (BalanceFactor(p) == -2 && BalanceFactor(p->rchild) > 0) {
        return RLRotation(p);
    }

    return p;
}

void Inorder(Node *p) {
    if (p == NULL)
        return;

    Inorder(p->lchild);
    printf("%d(h=%d,bf=%d) ", p->data, p->height, BalanceFactor(p));
    Inorder(p->rchild);
}

```

```

int main(void) {
    int keys[] = {50, 10, 20, 5, 2, 30, 40};
    int n = sizeof(keys) / sizeof(keys[0]);

    Node *root = NULL;

    for (int i = 0; i < n; i++) {
        root = Insert(root, keys[i]);
        printf("After inserting %d: ", keys[i]);
        Inorder(root);
        printf("\n");
    }

    return 0;
}

```

29. Understanding the Complete Code Step by Step

Step 1: Start With Empty Tree

```
Node *root = NULL;
```

The tree is empty.

Step 2: Insert Each Key

```
root = Insert(root, keys[i]);
```

This is important because insertion may return a new root after rotation.

Step 3: Insert Like BST

Inside `Insert`, the program compares the key:

```

if (key < p->data)
    go left
else if (key > p->data)
    go right

```


Step 4: Create Node at NULL Position

```
if (p == NULL)
    return CreateNode(key);
```

This creates the new leaf node.

Step 5: Return Upward

After insertion, recursion returns to the parent nodes.

Step 6: Update Height

```
UpdateHeight(p);
```

Step 7: Check Balance

```
BalanceFactor(p)
```

Step 8: Rotate If Needed

The program chooses LL, RR, LR, or RL.

Step 9: Return Local Root

Each rotation returns the new local root.

Finally, `Insert` returns the updated subtree root.

30. Dry Run: Inserting 50, 10, 20

This example creates an LR case.

Insert 50

```
50
```

Height:

```
height(50) = 1
BF(50) = 0
```

Balanced.

Insert 10

10 is smaller than 50, so it goes left.

```
  50
 /
10
```

Heights:

```
height(10) = 1
height(50) = 2
```

Balance factor of 50:

```
left height = 1
right height = 0
BF(50) = 1
```

Still balanced.

Insert 20

Start at 50:

```
20 < 50, go left
```

At 10:

```
20 > 10, go right
```

Tree becomes:

```
    50
   /
  10
   \
   20
```

Now update from bottom to top.

Node 20:

```
height = 1
BF = 0
```

Node 10:

```
left height = 0
right height = 1
height = 2
BF = -1
```

Node 50:

```
left height = 2
right height = 0
height = 3
BF = +2
```

Node 50 is imbalanced.

Now inspect left child 10:

```
BF(10) = -1
```

So:

```
Parent BF = +2
Left child BF = -1
Case = LR
```

Apply LR rotation.

After rotation:

```
    20
   /
  10   50
```

Now all nodes are balanced.

31. Dry Run: LL Rotation With Orphan Subtree

Suppose we have this tree:

```
      40
     /
    20
   /
  10  30
```

Here:

```
p = 40
p1 = 20
plr = 30
```

This is an LL-style rotation at `40`, but `20` already has a right subtree `30`.

Before:

```
      40
     /
    20
   /
  10  30
```

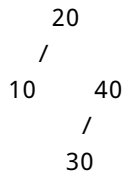
If `20` moves up and `40` moves down, where does `30` go?

Since:

```
20 < 30 < 40
```

30 must become the left child of 40.

After:



This is exactly why the code does:

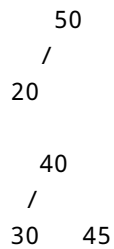
```
p->lchild = plr;
```

Memory line:

The orphan subtree is reattached where BST order says it belongs.

32. Dry Run: LR Rotation With Two Orphan Subtrees

Suppose we have this tree:



Here:

```
p = 50
pl = 20
plr = 40
```

The middle node is 40.

40 has two subtrees:

CL = 30
CR = 45

Before:

```
    50
   /
  20

    40
   /
  30  45
```

After LR rotation, 40 becomes the local root:

```
    40
   /
  20  50
 \  /
 30 45
```

Why?

Because:

20 < 30 < 40

So 30 becomes the right child of 20.

And:

40 < 45 < 50

So 45 becomes the left child of 50.

This is why the code does:

```
pl->rchild = plr->lchild;
p->lchild = plr->rchild;
```

Memory line:

In LR rotation, the middle node rises, and its two subtrees are reattached to the old lower nodes.

33. Time Complexity of AVL Insertion

AVL insertion takes:

$O(\log N)$

Why?

Because an AVL tree stays balanced, so its height is logarithmic.

Insertion has three parts:

1. Search for insertion position

This follows one path from root to leaf.

In AVL:

$\text{height} = O(\log N)$

So searching down takes:

$O(\log N)$

2. Update heights while returning

This also follows the same path upward.

So it takes:

$O(\log N)$

3. Rotation

A rotation changes only a fixed number of pointers.

So one rotation takes:

$O(1)$

Total:

$O(\log N)$

Memory line:

AVL insertion is $O(\log N)$ because the tree height stays $O(\log N)$.

34. Space Complexity of AVL Insertion

Recursive insertion uses the function call stack.

The number of recursive calls depends on tree height.

Since AVL height is logarithmic:

Space complexity = $O(\log N)$

Why?

Because recursion follows one path from the root to a leaf.

Memory line:

Recursive AVL insertion uses $O(\log N)$ stack space.

35. Time Complexity of One Rotation

One rotation takes:

$O(1)$

Why?

Because it only changes a fixed number of pointers.

It does not visit every node.

Example LL rotation changes only these pointers:

```
p1->rchild = p;  
p->lchild = plr;
```

Then it updates heights of only a few nodes.

So even if the full tree has 1,000,000 nodes, the rotation itself is still constant time.

Memory line:

Rotation is $O(1)$ because it does fixed work.

36. Common Beginner Mistakes in Phase 3

Mistake 1: Forgetting to Save Orphan Subtrees

Wrong idea:

Just move the nodes around without storing temporary pointers.

Problem:

A subtree can become disconnected and lost.

Correct idea:

Use temporary pointers like:

```
Node *plr = p1->rchild;  
Node *prl = pr->lchild;
```

Memory line:

Save before overwrite.

Mistake 2: Returning the Old Root After Rotation

Wrong idea:

```
return p;
```

after every rotation.

Problem:

After rotation, `p` may no longer be the root.

Correct idea:

Return the new root:

```
LL returns p1  
RR returns pr  
LR returns plr  
RL returns prl
```

Memory line:

Return the node that moved to the top.

Mistake 3: Updating Heights in the Wrong Order

Wrong idea:

Update the new root first.

Problem:

The new root depends on the heights of its children.

Correct idea:

Update lower nodes first.

Example LR:

```
UpdateHeight(pl);  
UpdateHeight(p);  
UpdateHeight(plr);
```

Memory line:

Lower nodes first, new root last.

Mistake 4: Checking Balance Before Updating Height

Wrong idea:

Check BF first.
Update height later.

Problem:

Balance factor may use old height values.

Correct idea:

Update height first.
Then check balance factor.

Mistake 5: Thinking Rotations Change Values

Wrong idea:

Rotation swaps values.

Correct idea:

Rotation changes pointers, not values.

The nodes move, but their data values stay inside the same nodes.

Memory line:

AVL rotations change links, not stored data.

Mistake 6: Thinking Rotations Break BST Order

Wrong idea:

Since the tree shape changes, the sorted order changes.

Correct idea:

Rotations preserve BST order.

The inorder traversal remains sorted.

Mistake 7: Not Handling Duplicate Keys Deliberately

In this beginner version, duplicate keys are ignored:

```
else {  
    return p;  
}
```

This means if the key already exists, the tree does not insert another copy.

That is okay as long as the rule is clear.

Memory line:

Choose a duplicate policy. This curriculum rejects duplicates.

37. Phase 3 Quick Reference Tables

AVL Node Fields

Field	Meaning
data	Stored key
height	Height of subtree rooted at this node
lchild	Left child pointer
rchild	Right child pointer

Balance Factor Meaning

BF	Meaning
+2	Left side too tall

BF	Meaning
+1	Left side slightly taller, valid
0	Equal height, valid
-1	Right side slightly taller, valid
-2	Right side too tall

Rotation Selection

Parent BF	Child BF	Case	New Root
+2	left child ≥ 0	LL	pl
+2	left child < 0	LR	plr
-2	right child ≤ 0	RR	pr
-2	right child > 0	RL	prl

Rotation Return Values

Rotation	Return Value	Why
LLRotation(p)	pl	Left child becomes root
RRRotation(p)	pr	Right child becomes root
LRRotation(p)	plr	Middle node becomes root
RLRotation(p)	prl	Middle node becomes root

38. Phase 3 Mental Model

Think of AVL insertion like a repair team.

The insertion goes down the tree like normal BST insertion.

Then it returns upward and asks each node:

1. What is my new height?
2. What is my balance factor?
3. Am I still balanced?
4. If not, which rotation fixes me?

If a rotation is needed, the program carefully moves pointers.

It does not lose subtrees.

It does not break BST order.

It returns the new root of the repaired subtree.

39. Step-by-Step Method to Trace Phase 3 Insertion

When tracing AVL insertion by hand, use this method.

Step 1: Insert Like a BST

Use the rule:

```
smaller goes left
greater goes right
```

Step 2: Start From the Inserted Node

The new node has:

```
height = 1
BF = 0
```

Step 3: Move Upward

For each ancestor, calculate:

```
height = 1 + max(left height, right height)
BF = left height - right height
```

Step 4: Find Imbalance

If BF becomes `+2` or `-2`, the node is imbalanced.

Step 5: Check Child Balance

Use the child balance factor to choose LL, RR, LR, or RL.

Step 6: Apply Rotation

Use the correct rotation.

Step 7: Reattach Orphan Subtrees

Make sure every subtree still belongs in the correct BST position.

Step 8: Update Heights

Update lower nodes first, then the new root.

Step 9: Return New Root

The rotated subtree must return its new root.

40. Practice Example 1: Identify the Rotation

Tree after insertion:

```
    50
   /
  10
   \
   20
```

Find the imbalanced node:

50 has BF = +2

Check left child:

10 has BF = -1

Decision:

Parent BF = +2
Left child BF = -1
Case = LR

Fix:

LR rotation

Result:

```
    20
   /
  10   50
```

41. Practice Example 2: Orphan Subtree in LL

Before:

```
    40
   /
  20
 /
10  30
```

Identify nodes:

```
p = 40
pl = 20
plr = 30
```

After LL rotation:

```
    20
   /
  10   40
       /
      30
```

Important point:

30 becomes left child of 40

because:

20 < 30 < 40

42. Practice Example 3: Orphan Subtrees in LR

Before:

```
      50
     /
    20

      40
     /
    30  45
```

After LR rotation:

```
      40
     /
    20  50
   \  /
  30 45
```

Important reattachments:

30 becomes right child of 20
45 becomes left child of 50

Why?

20 < 30 < 40
40 < 45 < 50

43. Self-Test Questions

Question 1

Why does an AVL node store height?

Answer:

So the program can calculate balance factor quickly in $O(1)$ time.

Question 2

What is the balance factor formula?

Answer:

$$BF = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$$

Question 3

Why must we check for `NULL` before reading child height?

Answer:

Because trying to access a missing child causes an invalid pointer access.

Question 4

When does AVL insertion update height?

Answer:

After the recursive insertion returns, while moving back up the tree.

Question 5

Why do we update height before checking balance factor?

Answer:

Because balance factor depends on the current child heights.

Question 6

If parent BF is `+2` and left child BF is `-1`, which case is it?

Answer:

LR case.

Question 7

If parent BF is `-2` and right child BF is `+1`, which case is it?

Answer:

RL case.

Question 8

In LL rotation, which node becomes the new root?

Answer:

The left child, called `p1` in code.

Question 9

In LR rotation, which node becomes the new root?

Answer:

The middle node, called `plr` in code.

Question 10

Why must rotations return a pointer?

Answer:

Because the root of the local subtree may change after rotation.

Question 11

What is an orphan subtree?

Answer:

It is a subtree that could get disconnected during rotation if we do not save and reattach it.

Question 12

Do rotations change the sorted order of the tree?

Answer:

No. Rotations change shape, not sorted order.

Question 13

What is the time complexity of one rotation?

Answer:

$O(1)$

Question 14

What is the time complexity of AVL insertion?

Answer:

$O(\log N)$

44. One-Sentence Memory Lines

Phase 1 detects imbalance.
Phase 2 explains rotation shapes.
Phase 3 implements rotations safely in C.

AVL insertion starts like BST insertion, then fixes balance while returning upward.

Update height before checking balance factor.

Save orphan subtrees before overwriting pointers.

Rotations change pointers, not values.

Rotations change shape, not sorted order.

Every rotation returns the new local root.

Update lower nodes first, then update the new root.

LL and RR are mirror rotations.

LR and RL are mirror rotations.

In zig-zag cases, the middle node moves up.

45. Final Phase 3 Summary

Phase 3 teaches how AVL rotations are implemented in C.

An AVL node stores:

```
left child pointer
data
height
right child pointer
```

The `height` field lets the program calculate balance factor quickly.

The balance factor formula is:

```
BF = height(left subtree) - height(right subtree)
```

AVL insertion first behaves like BST insertion:

```
smaller values go left  
greater values go right
```

After inserting the new node, recursion returns upward.

During this return phase, the program:

```
updates height  
checks balance factor  
chooses rotation if needed  
returns the new subtree root
```

The four rotation cases are:

```
LL, RR, LR, RL
```

The rotation is selected by checking:

```
parent balance factor  
child balance factor
```

In large trees, rotations must handle attached subtrees.

These attached subtrees are called orphan subtrees when they might get disconnected during pointer movement.

The rule is:

```
Save orphan subtrees before overwriting pointers.
```

Rotations preserve BST order because every moved subtree is reattached where its values still satisfy:

```
left < node < right
```

A rotation takes:

$O(1)$

because it changes only a fixed number of pointers.

AVL insertion takes:

$O(\log N)$

because AVL trees stay balanced and their height remains logarithmic.

46. Phase 3 Mastery Checklist

You are ready for Phase 4 if you can:

- Explain why AVL nodes store height.
- Write or understand the AVL node structure in C.
- Explain `lchild`, `rchild`, `data`, and `height`.
- Explain the `NodeHeight` function.
- Explain the `BalanceFactor` function.
- Remember that NULL child height is 0.
- Remember that leaf height is 1.
- Explain why insertion starts like BST insertion.
- Explain why AVL insertion uses recursion.
- Explain why repair happens while recursion returns.
- Update height before checking balance factor.
- Use parent and child balance factors to choose LL, RR, LR, or RL.
- Explain what an orphan subtree is.
- Explain why orphan subtrees must be saved.
- Trace LL rotation with an attached subtree.
- Trace RR rotation with an attached subtree.
- Trace LR rotation with two attached subtrees.
- Trace RL rotation with two attached subtrees.
- Explain why rotations preserve BST order.
- Explain why each rotation returns the new root.
- Explain why rotation is $O(1)$.
- Explain why AVL insertion is $O(\log N)$.

If you can do these, you understand Phase 3 well.

47. Final Memory Line

Phase 3 is about turning AVL rotation drawings into safe C pointer code without losing subtrees or breaking BST order.